

```

// Free Pointer : Tells where the new data will be stored
// Start Pointer : Tells where the linked list is starting from

TYPE linkedList
    DECLARE Node : STRING
    DECLARE NextPointer : INTEGER
ENDTYPE

DECLARE StartPointer , FreePointer : INTEGER
DECLARE List : ARRAY [1:5] OF linkedList

CONSTANT NullPointer = 0

// INITIALIZING OF LINKED LIST

PROCEDURE Initialise()
    StartPointer <- 0
    FreePointer <- 1
    FOR x = 1 TO 4
        list[x].NextPointer <- x + 1
    NEXT x

list[5].NextPointer <- 0

PROCEDURE AddItem():
    IF FreePointer == NullPointer THEN
        OUTPUT "Linked list is full"
    ELSE
        Data = INPUT("Enter Item")
        List[FreePointer].Node = Data
        IF StartPointer == NullPointer THEN // if it is the first data
            StartPointer <- FreePointer // setting the starting location of the list
            Temp <- list[FreePointer].NextPointer // storing it cause we have to add this to the Free Pointer later
            list[FreePointer].NextPointer <- NullPointer
            FreePointer <- Temp
        ELSE
            Temp <- list[FreePointer].NextPointer // if it is the smallest data
            IF Data < List[StartPointer].Node THEN // comparing the data
                List[FreePointer].NextPointer <- StartPointer // changing the right pointer of the FP to the before start item
                StartPointer <- FreePointer
                FreePointer <- Temp
            ENDIF
        ELSE // if middle data
            PreviousPointer <- NullPointer
            CurrentPointer <- StartPointer
            Found <- False
            REPEAT
                IF Item <= List[CurrentPointer].Node THEN
                    List[PreviousPointer].NextPointer <- FreePointer
                    List[FreePointer].NextPointer <- CurrentPointer
                    Found <- True
                    FreePointer <- Temp
                ELSE
                    PreviousPointer <- CurrentPointer
                    CurrentPointer <- List[CurrentPointer].NextPointer
                ENDIF
            UNTIL Found = True OR CurrentPointer = NullPointer
            // If the item is largest
            IF Found = False THEN
                List[PreviousPointer].NextPointer <- FreePointer
                List[FreePointer].NextPointer <- NullPointer
                FreePointer <- Temp
            ENDIF
        ENDIF
    ENDIF

// DELETE NODE
PROCEDURE DeleteItem(item : STRING)
    Found <- False
    CurrentPointer <- StartPointer

    WHILE CurrentPointer <> NullPointer AND Found = False
        IF item = MyList[CurrentPointer].Node THEN
            Found <- False
        ELSE
            PreviousPointer <- CurrentPointer
            CurrentPointer <- MyList[CurrentPointer].NextPointer
        ENDWHILE

    IF CurrentPointer <> NullPointer THEN
        IF CurrentPointer = StartPointer THEN
            StartPointer <- MyList[CurrentPointer].NextPointer
        ELSE
            MyList[PreviousPointer].NextPointer <- MyList[CurrentPointer].NextPointer
        ENDIF
    ENDIF

```

```
ENDIF  
  
  MyList[CurrentPointer].NextPointer <- FreePointer  
  FreePointer <- CurrentPointer  
ENDIF  
END
```